AI – ASSISTED CODING

2303A510A0 – VISHNU VARDHAN

Assignment – 20.1

Batch – 14

Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code

Task 1 – Input Validation Check

PROMPT:

Generate a simple Python login program that takes username and password from user input and checks them with stored values.Do not include input validation.

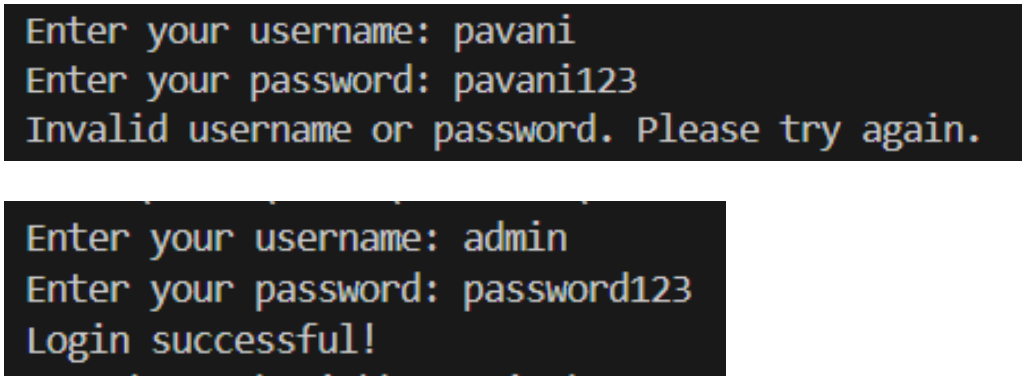
CODE:

```
stored_username = "admin"
stored_password = "password123"
# Function to check login credentials
def login():
    # Get username and password from user input
    username = input("Enter your username: ")
    password = input("Enter your password: ")

    # Check if the entered credentials match the stored values
    if username == stored_username and password == stored_password:
        print("Login successful!")
    else:
        print("Invalid username or password. Please try again.")

# Call the login function
login()
```

OUTPUT:



EXPLANATION:

1. The login program takes username and password without checking their format.
2. This allows invalid or malicious inputs (like special characters or scripts).
3. Lack of validation can lead to security issues like injection attacks.
4. Input validation ensures data follows specific rules (length, characters, etc.).
5. Using re (regular expressions) improves security by restricting allowed input patterns.

Task 2 – SQL Injection Prevention

PROMPT:

Generate a Python script using SQLite that fetches user details from a database using username input.Use simple string concatenation in the SQL query.

CODE:

```
import sqlite3
# Connect to the SQLite database (or create it if it doesn't exist)
conn = sqlite3.connect('users.db')
# Create a cursor object to interact with the database
cursor = conn.cursor()
# Create a table to store user details (if it doesn't already exist)
cursor.execute('''CREATE TABLE IF NOT EXISTS users (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,

                    username TEXT NOT NULL,
                    password TEXT NOT NULL,
                    email TEXT NOT NULL)''')
# Insert a sample user into the database (for testing purposes)
cursor.execute("INSERT INTO users (username, password, email) VALUES ('admin', 'password123', 'admin@example.com')")
# Commit the changes to the database
conn.commit()
# Function to fetch user details based on username input
def fetch_user_details():
    # Get username from user input
    username = input("Enter the username to fetch details: ")
    # Create a SQL query using string concatenation (not recommended for production code)
    query = "SELECT * FROM users WHERE username = '" + username + "'"
    # Execute the query
    cursor.execute(query)
    # Fetch the user details
    user_details = cursor.fetchone()
    # Check if user details were found and print them
    if user_details:
        print("User Details:")
        print("ID:", user_details[0])
        print("Username:", user_details[1])
        print("Password:", user_details[2])
        print("Email:", user_details[3])
    else:
        print("No user found with the username:", username)
# Call the function to fetch user details
fetch_user_details()
# Close the database connection
conn.close()
```

OUTPUT:

```
Enter the username to fetch details: admin
User Details:
ID: 1
Username: admin
Password: password123
Email: admin@example.com
```

EXPLANATION:

1. The program builds SQL queries using string concatenation.
2. This allows attackers to manipulate queries using malicious input.
3. Example: ' OR '1'='1 can bypass authentication.
4. This vulnerability is called SQL Injection.

5. Using parameterized queries (?) prevents injection by separating data from query.

Task 3 – Cross-Site Scripting (XSS) Check

PROMPT:

Convert this SQL query into an equivalent Pandas DataFrame operation in Python.

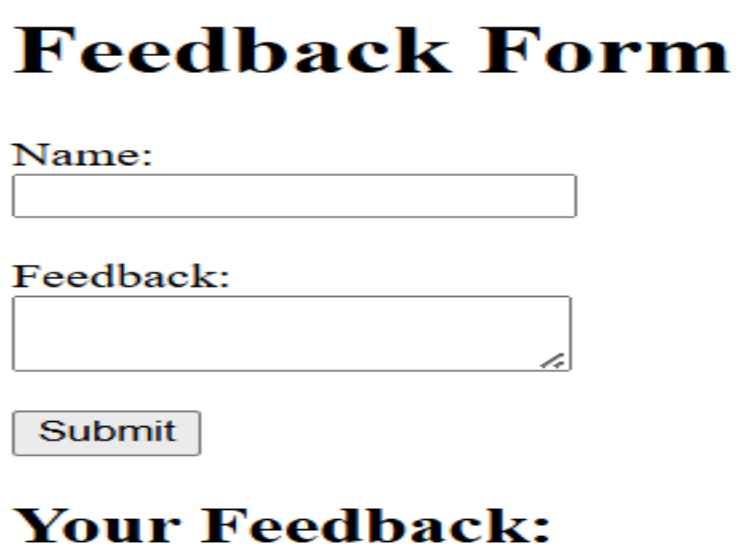
CODE:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Feedback Form</title>
</head>
<body>
    <h1>Feedback Form</h1>
    <form id="feedbackForm">
        <label for="name">Name:</label><br>
        <input type="text" id="name" name="name"><br><br>
        <label for="feedback">Feedback:</label><br>
        <textarea id="feedback" name="feedback"></textarea><br><br>
        <button type="button" onclick="submitFeedback()">Submit</button>
    </form>

    <h2>Your Feedback:</h2>
    <div id="displayFeedback"></div>

    <script>
        function submitFeedback() {
            var name = document.getElementById('name').value;
            var feedback = document.getElementById('feedback').value;
            var displayDiv = document.getElementById('displayFeedback');
            displayDiv.innerHTML = "<p><strong>Name:</strong> " + name + "</p><p><strong>Feedback:</strong> " + feedback + "</p>";
        }
    </script>
</body>
</html>
```

OUTPUT:



EXPLANATION:

1. The web form displays user input directly using innerHTML.
2. This allows execution of malicious scripts entered by users.
3. Example: <script>alert('Hacked')</script> runs in browser.
4. This vulnerability is called XSS (Cross-Site Scripting).
5. Using textContent or escaping input prevents script execution.

Task4:Real-Time App: Security Audit of AI-Generated Code

PROMPT:

Generate a simple web application login system (frontend + backend) with basic authentication.

CODE:

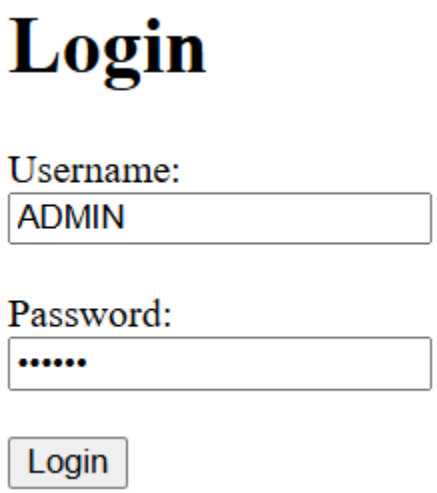
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login System</title>
</head>
<body>
    <h1>Login</h1>
    <form id="loginForm">
        <label for="username">Username:</label><br>
        <input type="text" id="username" name="username"><br><br>
        <label for="password">Password:</label><br>
        <input type="password" id="password" name="password"><br><br>
        <button type="button" onclick="login()">Login</button>
    </form>

    <div id="loginResult"></div>

    <script>
        function login() {
            var username = document.getElementById('username').value;
            var password = document.getElementById('password').value;

            fetch('/login', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                },
                body: JSON.stringify({ username: username, password: password })
            })
            .then(response => response.json())
            .then(data => {
                var resultDiv = document.getElementById('loginResult');
                if (data.success) {
                    resultDiv.innerHTML = "<p>Login successful! Welcome, " + data.username + ".</p>";
                } else {
                    resultDiv.innerHTML = "<p>Login failed. Please try again.</p>";
                }
            });
        }
    </script>
</body>
</html>
```

OUTPUT:



Task 5: Insecure Data Serialization Check

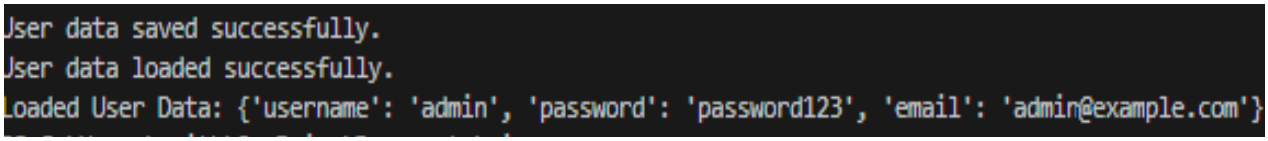
PROMPT:

Generate a Python script that saves and loads user data using pickle.

CODE:

```python
import pickle
# Function to save user data to a file using pickle
def save_user_data(user_data, filename):
    with open(filename, 'wb') as file:
        pickle.dump(user_data, file)
    print("User data saved successfully.")
# Function to load user data from a file using pickle
def load_user_data(filename):
    try:
        with open(filename, 'rb') as file:
            user_data = pickle.load(file)
        print("User data loaded successfully.")
        return user_data
    except FileNotFoundError:
        print("File not found. Returning an empty dictionary.")
        return {}
# Example usage
if __name__ == "__main__":
    # Sample user data to save
    user_data = {
        'username': 'admin',
        'password': 'password123',
        'email': 'admin@example.com'
    }
    # Save the user data
    save_user_data(user_data, 'user_data.pkl')
    # Load the user data
    loaded_data = load_user_data('user_data.pkl')
    print("Loaded User Data:", loaded_data)
```

OUTPUT:



EXPLANATION:

1. The program uses pickle to store and load data.
2. pickle.load() can execute malicious code if input is untrusted.
3. This makes it unsafe for handling external data.
4. Attackers can exploit it to run harmful commands.
5. Using JSON is safer as it only stores data, not executable code.